NYCU 國立陽明交通大學
自主及適應性系統實驗室
Autonomous and Adaptive Systems Laboratory

# Homework Assignment 1: Games for Graph Problems

**CSIC30122: Games Theory and Its Applications**

Professor: Li-Hsing Yen
TA: YAN-WEI CHEN, CHUN-YU LIN

**DeadLine: 2026-0407-2359**

*Institute of Computer Science and Engineering*
*National Yang Ming Chiao Tung University*

AASLAB

# Outline

- Overview
- Graph Problems of Assignment
  - Problem set 1
  - Problem set 2
  - Problem set 3
- Constructing WS Model
- Requirement
- Scoring Policy
  - Submission
- References

# Overview

- Solving graph problems with game theoretic approach
  - There are three graph problem sets
  - Each part includes code writen, test cases, and result analysis

- Generating WS Model graphs to evaluate game approach
  - Test cases for graph problems

Summarizing into one report: briefly describe your code and show the test cases as well as result analysis

# How to Solve Graph Problems via Game?

Repeating (for each player)

- Choosing the best response according to the observation of other players' strategy until arrival Nash Equilibrium

*"Nash Equilibrium: a strategy profile (game state) where no player can further increase its own utility by unilaterally changing its strategy."*

# Pseudo Programming Logic

```
randomize initial game state
move_count = 0
while the game does not reach NE
  randomly pick up one player who can improve its utility
  change this player's strategy to its best response
  move_count++
end while
verify that the game state is a valid solution
output game state and move_count
```

# Goal of Graph Problems via Game

- Except weighted MIS game, results can be evaluated by the number of elements in the selected set
  - For dominating set, we want to **minimize** the elements
  - For independent set, we try to **maximize** the elements
- With respect to weighted MIS, we seek to **maximize** the <u>total weight</u> of the selected set

- Moreover, we should also pay close attention to the number of players' movements (i.e., `move_count`)

# Graph Problems - Set 1

## Selecting one graph game based on your student ID

| Student ID mod 5 | Game to simulate | Reference |
|---|---|---|
| 0 | MIS-based IDS Game | Lec. graph_games p. 62 |
| 1 | $k$-Domination Game (***k*=2**) | Lec. graph_games p. 25 |
| 2 | **Symmetric** Maximal Independent Set (MIS) Game | Lec. graph_games p. 33 |
| 3 | **Asymmetric** MIS Game (nodes with lower node degrees have higher priority) | Lec. graph_games p. 49 |
| 4 | Weighted MIS Game (each node has a unique integer weight set as node_id, i.e., node_1=1, node_2=2, …) | Lec. graph_games p. 55 |

Please comply with the red-word rules

# Graph Problems - Set 2

## Selecting one graph game based on your student ID

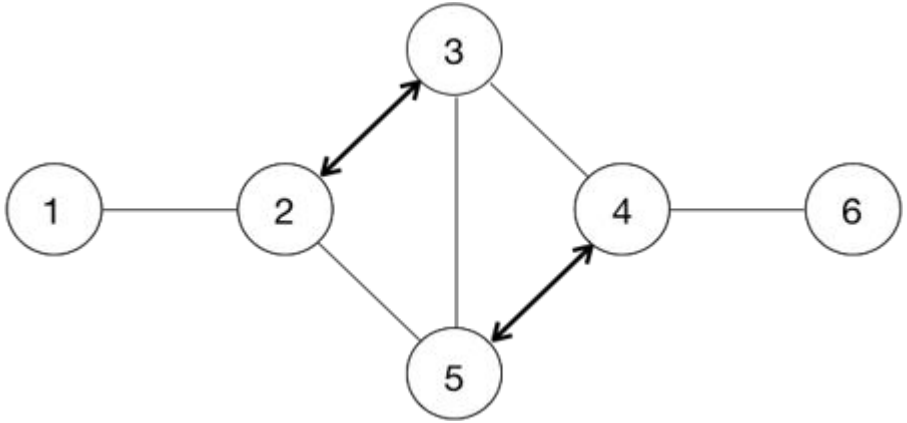
| Student ID mod 2 | Game to simulate | Notes |
|---|---|---|
| 0 | **Symmetric** MDS-based IDS Game | Lec. graph_games p. 65 |
| 1 | **Asymmetric** MDS-based IDS Game (nodes with higher node degrees have higher priority) | Lec. graph_games p. 68 |

Please comply with the red-word rules

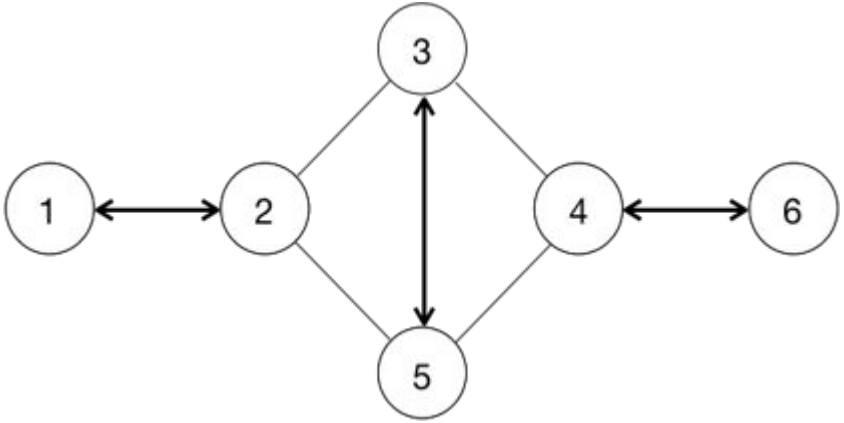
# Graph Problems - Set 3

**Maximal** Matching: https://en.wikipedia.org/wiki/Matching_(graph_theory)

- Defining your own utility function for reaching maximal matching




a valid maximal matching




another valid maximal matching

# Matching Game

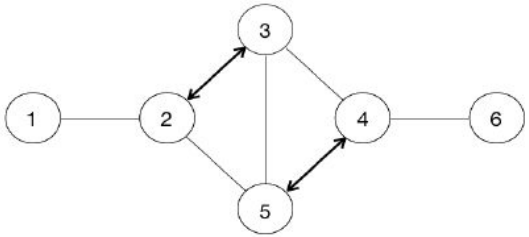
- Given a graph $G = (P, E)$, where $P$ is the vertex set (with $n$ vertices) while $E$ is the edge set
- Each player $p_i$ is a node in $P$
- Strategy set of each player $p_i \in P$: $N_i \cup \{null\}$, where $N_i$ is $p_i$'s open neighbors and $null$ indicates unmatched
- Let $c_i$ be $p_i$'s strategy and $C = (c_1, c_2, \cdots, c_n)$ be a strategy profile. $(p_i, p_j)$ is a *matched pair* if and only if $c_i = p_j$ and $c_j = p_i$.
- $p_i$ is *unmatched* if $c_i = null$

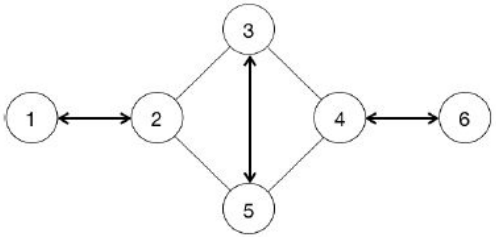
# Design Goal

- Define a utility function $u_i(C)$ for each player $p_i \in P$ such that
  - Starting from any strategy profile, ensure that any player's strategy change as his best response eventually ends up with a NE
  - Ensure that every NE is a maximal matching
    - It's a matching because ① $c_i = p_j$ whenevr $c_j = p_i$ ② $c_i = null$ otherwise
    - It's maximal because there exists no two players that are unmatched but could match with each other

# Design Goal (Optional)

- A heuristic to increase the number of matched pairs is to give high priority to nodes with few neighbors in matching
- Example




- Try to integrate this concept into utility definition
- This part is not mandatory (only for those who are interested)

# Watts-Strogatz (WS) Model

WS model [WS98] can help you evaluate your approach

- Generalization and scalability of your game approach
  - Randomness of WS model

You need to generate WS model to simulate

- Showing in report

Hidden Test Case also includes a WS model generated by TAs

# WS Model Construction

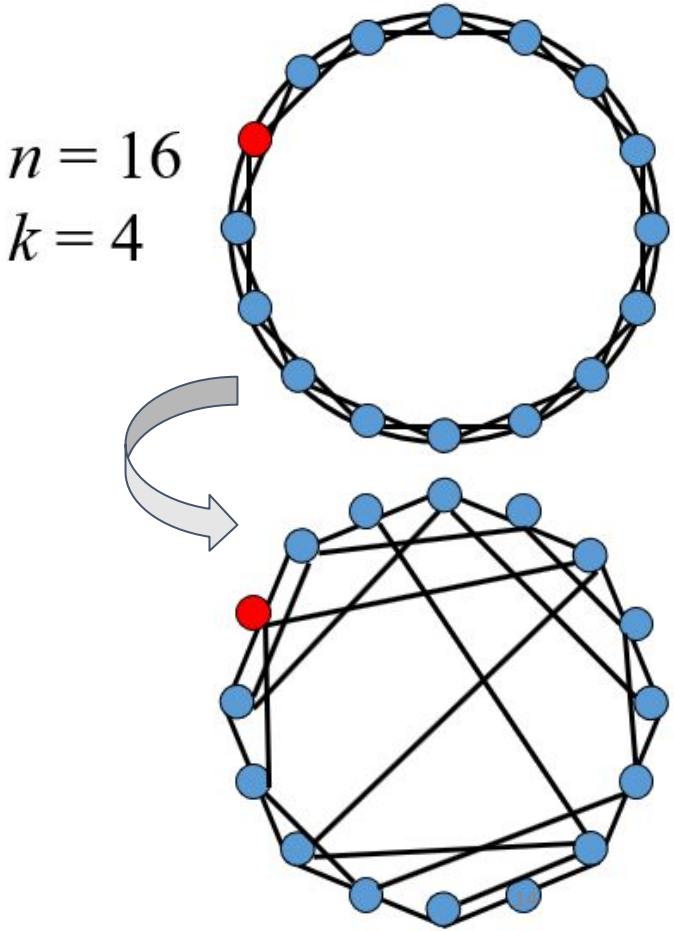
An n-node regular graph is first constructed

- Each node has **k** edges connecting to its **k** nearest neighbors

Rewiring every edge to a randomly selected node with probability *p*

*Each node in WS model has at least one edge connect to others*

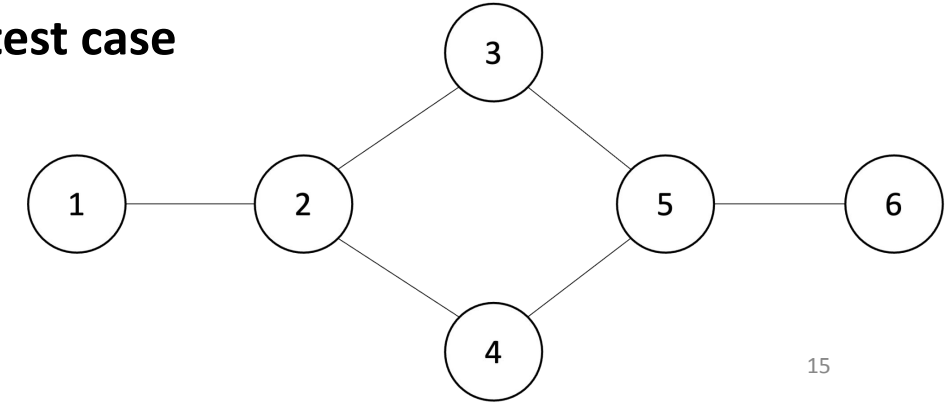
*~ i.e.,* ***no isolated node***



# Input Format and Given Test Case

Graph representation:

- 6 010000 101100 010010 010010 001101 000010
  - The first number is the number of nodes k
  - The following k k-bit streams represent nodes' relations with other nodes respectively, i.e., relations of node1, then relations of node2, and so on
    - "1" indicates an edge of the corresponding two nodes
  - **This example is also the given test case**



# Additional Requirement

Hence, TAs will **feed a string to your program to check the results**

- Command-line arguments: the number of nodes and relations among nodes

**Should print your cardinality** of set for each selected graph problems

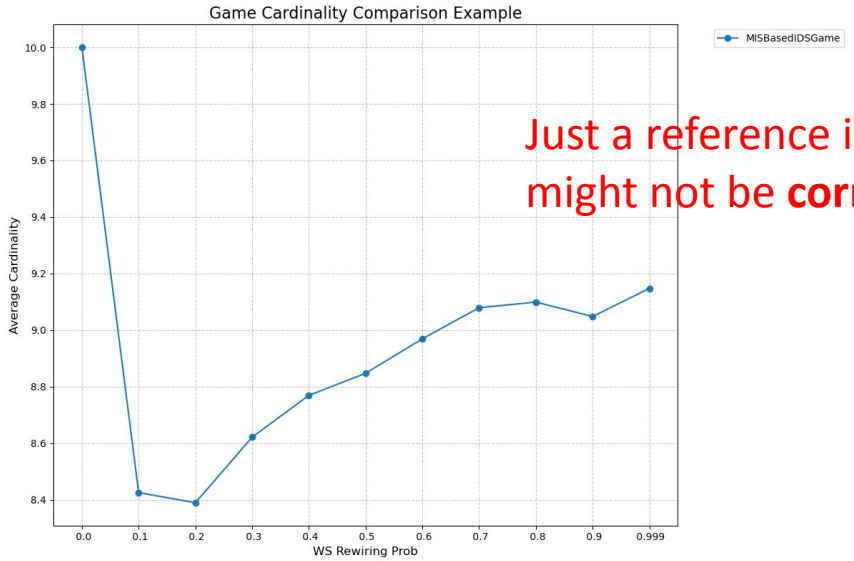
Here are solutions for given test case

```
Requirement 1-1:
the cardinality of MIS-based IDS Game is 2
the cardinality of 2-dominating game is 4
the cardinality of Maximal Independent Set (MIS) Game is 4
the cardinality of Asymmetric MIS Game is 4
the cardinality of Weighted MIS Game is 4
Requirement 1-2:
the cardinality of Symmetric MDS-based IDS Game is 2
the cardinality of Asymmetric MDS-based IDS Game is 2
Requirement 2:
the cardinality of Matching Game is 2
```

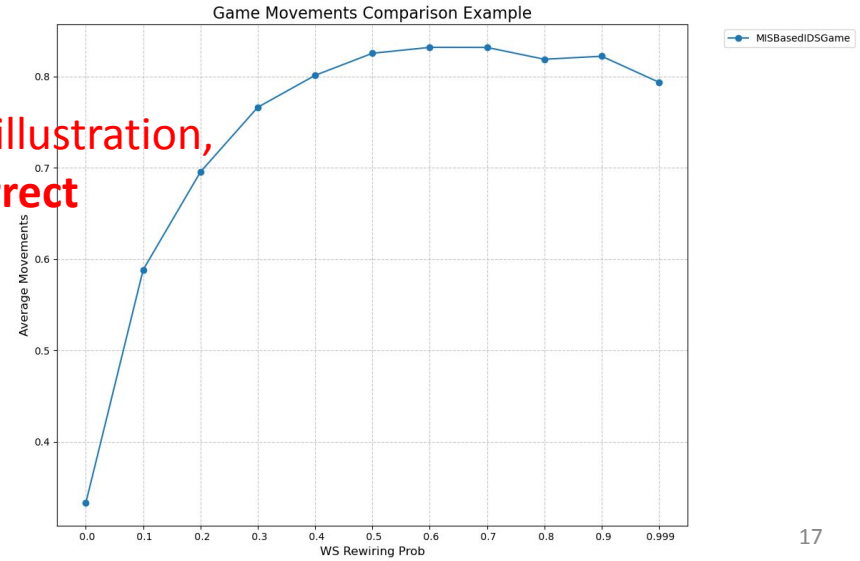
# Additional Requirement

Performance evaluation of your method for graph problems

- e.g., conducting experiments of avg. cardinality/movement according to WS node number/rewiring probability……




Just a reference illustration, might not be **correct**



# Additional Requirement

- Make sure your program can parse arguments
- Make sure your program can execute successfully
- Third-party packages or modules is not allowed
  - ~~Directly invoking methods like maximum_matching (networkX) to solve~~
  - **Implementing by yourself**

If the answer is not **unique**, please output **extremum** value(maximum or minimum value) of candidate answers based on the execution

# Scoring Policy

**100 points =**

- ✓ Problem Set 1 (**15 points**): 6 for extreme case, 9 for hidden one
- ✓ Problem Set 2 (**15 points**): 9 for extreme case, 6 for hidden one
- ✓ Problem Set 3 (**30 points**): 10 for extreme case, 20 for hidden one
- ✓ Report (**40 points**): code description, all test cases you use (involving given one), result analysis, and **AI prompt (if you use)**
  - Written in either <u>English or Mandarin</u>

Hidden case: a WS graph (node # ≤ 30; w/o isolated node; identical input format)

# Scoring Policy

Report of each requirement will be classified into one of the five tiers:

- Excellent (40 points); good (32 points); normal (25 points); nonsensical result and analysis (15 points); no answer (0 points)

**NO POINT:**

- **Plagiarism in source code or report**
- **Directly invoking methods like maximum_matching to solve**

Strict 30% off deduction for **Late Turning In**

~ No matter when after the deadline

# Submission

Submitting a zip file to E3 platform with following structure:

✓ <your student ID>_HW1.zip (e.g., 412551007_HW1.zip)

- ➢ <your student ID>_HW1_report.pdf
- ➢ src/
  - <your student ID>_HW1_main.py/.c/.cpp
  - ……(optional, other necessary file)

Strict 10% off deduction for **Wrong Format**

# References


- [WS98] D.J. Watts and S.H. Strogatz, “Collective Dynamics of ‘Small-World’ Networks,” Nature, vol. 393, pp. 440-442, June 1998.
- [YC14] L.-H. Yen and Z.-L. Chen, “Game-theoretic approach to self-stabilizing distributed formation of minimal multi-dominating sets,” IEEE Trans. on Parallel and Distributed Systems, 25(12): 3201-3210, Dec. 2014.
- [YHT16] L.-H. Yen, J.-Y. Huang, and V. Turau, “Designing self-stabilizing systems using game theory,” ACM Trans. on Autonomous and Adaptive Systems, 11(3), Sept. 2016.
- [YS18] L.-H. Yen and G.-H. Sun, “Game-theoretic approach to self-stabilizing minimal independent dominating sets,” The 11th Int'l Conf. on Internet and Distributed Computing Systems (IDCS 2018), Tokyo, Japan, Oct. 2018.
- https://en.wikipedia.org/wiki/Matching_(graph_theory)